3.X Front-End Architecture & Implementation

This section details the architectural paradigm, visual engineering, state mechanics, network synchronization, and offline capabilities of the VulnSneak-AI client-side application. The front-end is engineered as a highly responsive, performance-optimized Single Page Application (SPA), designed to facilitate autonomous security analysis, interact with dynamic AI models, and render complex visual data with low latency within client-side environments.

3.X.1 Architectural Paradigm and Technical Stack

The client-side infrastructure of VulnSneak-AI is built around the principles of modularity, performant rendering, and secure state separation. The development stack carefully balances rich visual feedback with the efficient processing of static security code inputs.

Core UI Library: React 18.2 serves as the baseline declarative UI library, leveraging concurrent rendering features to handle complex DOM updates during real-time code scanning operations without blocking user interaction.

Build Tool & Bundler: Vite 7.2 is adopted as the primary bundler and development server. Its architecture enables fast Hot Module Replacement (HMR) during development and produces highly optimized production builds through its underlying Rollup pipeline.

Styling Architecture: Tailwind CSS v4 is employed to enforce a utility-first styling discipline. The framework compiles directly to modern CSS variables, maintaining clean and consistent theme configurations across all layout contexts.

State Decoupling & Optimization: The React Context API is used to manage global application states, including authentication and theme preferences, eliminating the need for heavier third-party state management libraries and reducing overall bundle size.

Code Splitting: Heavy dashboard structures are loaded on demand using deferred component loading techniques, ensuring that the initial page load remains lightweight and that First Contentful Paint (FCP) metrics are kept within acceptable thresholds.

3.X.2 UI/UX Motion Mechanics and Shader Pipelines

To deliver a high-fidelity security console experience, VulnSneak-AI integrates dedicated client-side rendering engines for fluid graphics, interactive cursor behavior, and physics-driven animations.

A. Liquid Chrome Shader System (LiquidChrome.jsx)

The platform implements a WebGL-based fluid dynamics background using the lightweight OGL graphics framework. The rendering pipeline is driven by custom GLSL

shaders that are compiled at runtime directly on the client's GPU.

The vertex shader is responsible for mapping two-dimensional screen coordinates and UV spaces to the normalized drawing canvas. The fragment shader performs intensive per-pixel calculations to simulate metallic, chrome-like ripple distortions. It operates by applying iterative multi-directional wave transformations across the UV coordinate space, driven by parameters including elapsed time, screen resolution, and normalized mouse position. The result is a continuously animated, fluid surface that responds dynamically to user cursor movement, creating an immersive visual environment without any reliance on pre-rendered assets or video files.

B. Antigravity 3D Particle Engine (Antigravity.jsx)

Integrated through the React Three Fiber (R3F) declarative wrapper, this component manages a three-dimensional particle system of up to 300 instances — including capsule, sphere, and box geometries — within a single shared WebGL context.

To maintain rendering performance and minimize GPU draw calls, the system uses instanced mesh rendering, where all particle instances share a single geometry buffer and their individual positions are updated through matrix transformations applied on the GPU rather than through separate draw operations.

The system also implements interactive force field behavior. During every animation frame, the engine calculates the spatial distance between each particle and the projected cursor position on the viewport plane. Particles that enter a defined magnetic radius have their target trajectories recalculated along a circular orbit path, producing an interactive magnetic ring effect around the cursor that responds in real time to user movement.

C. GSAP Animation and ScrollTrigger Synchronization

ScrollTrigger Layout Correction: In Single Page Applications, client-side route transitions can silently shift DOM element positions, causing scroll-driven animation trigger points to become misaligned. VulnSneak-AI resolves this by binding a recalibration callback to a custom page transition event. Upon each navigation completion, a recalculation of all active scroll triggers is requested within the next available animation frame, ensuring geometric accuracy across all page layouts.

CardSwap Stack Mechanics (CardSwap.jsx): This component uses custom GSAP timelines to cycle a stack of cards through a three-dimensional perspective space. Each card transition dynamically adjusts both depth positioning along the Z-axis and stacking order, producing a seamless elastic cycling effect without any card overlap or visual discontinuity.

D. Interactive Custom Cursor (CustomCursor.jsx)

The platform replaces the native browser cursor with a dual-node custom implementation consisting of a precision central dot and a larger, intentionally lagging outer ring.

The central dot tracks the true mouse position with zero delay. The outer ring follows with a configurable lag factor, calculated on every tick of the animation loop by interpolating the ring's current position toward the mouse position by a fixed fraction. This interpolation produces a smooth, physically plausible trailing motion.

A MutationObserver continuously monitors the live DOM subtree for newly rendered interactive elements such as buttons, links, and inputs. When such elements appear, cursor interaction callbacks are dynamically bound to them without requiring manual re-registration, ensuring consistent cursor behavior even as the UI updates asynchronously.

3.X.3 Application Routing, Pages, and Interactive Modules

The application is organized into ten core pages managed through a declarative client-side routing structure:

No.	Route	Component	Access Type	Description
1	/	Home	Public	Core landing page with lazy-loaded sections and bento-style highlights.
2	/auth	AuthPage	Public	Secure entry gateway featuring an interactive 3D-tilting authorization card.
3	/info	ProfileEdit	Protected	Management dashboard for user credentials, passwords, and data deletion.
4	/ai	SplineAgentPage	Protected	Main interactive scanning interface with 3D model overlays and file ingestion.
5	/about	About	Public	Team registry showcasing project contributors with GSAP mouse-masking layers.
6	/faq	FAQ	Public	Interactive Q&A answering common architecture and methodology inquiries.
7	/doc	Docs	Public	Technical documentation repository with sticky navigation and scroll-spy indexing.
8	/blog	BlogPage	Public	Knowledge-base featuring write-ups on critical web vulnerability classes.
9	/contact	ContactUs	Public	Secure feedback portal integrated with third-party form delivery endpoints.
10	*	Page404	Public	Dynamic 404 page with an animation loop simulating friction on mouse coordinate offsets.

Major Interactive Components

Autonomous Scan Dashboard (SplineAgentPage & NeuralCodeIntelligence): This component serves as the primary interface for vulnerability analysis. It displays detected vulnerability data, code differences between original and repaired versions, and step-by-step remediation guidance. Code rendering is handled by a custom syntax tokenizer built from scratch to avoid the overhead of external parsing libraries. The tokenizer categorizes code tokens — including keywords, string literals, numeric values, comments, and function identifiers — using pattern matching, and presents them in a split-screen side-by-side layout comparing the original vulnerable code against the AI-generated repaired version. Lines identified as vulnerable are rendered with a distinctive animated highlight to draw immediate developer attention.

System Architecture Visualization (FlowchartSection.jsx): To support developer understanding of the system pipeline, this component renders an interactive architectural diagram within a styled glassmorphic frame. A hover-triggered overlay launches a full-screen lightbox modal with smooth entrance and exit transitions, enabling users to inspect the full data flow pipeline — spanning the frontend, secure API gateway, Raspberry Pi proxy, and AI inference engines — at high resolution without leaving the current page context.

3.X.4 Data Flow, Validation, and Client-Side Security

Data integrity and privacy are enforced at the client-side boundary through carefully designed state control and input validation procedures.

Theme State Preservation (ThemeContext.jsx): User interface preferences, specifically light and dark mode selections, are persisted in the browser's local storage under a dedicated application key. The context provider propagates the active theme by modifying the document root element's class list, which in turn activates the corresponding set of CSS utility variables throughout the entire component tree without requiring any component-level re-renders.

Identity and Token Isolation (AuthContext.jsx): To mitigate the risk of token leakage through Cross-Site Scripting (XSS) attacks, authentication tokens and user metadata are stored exclusively within session storage rather than local storage or in-memory JavaScript variables. This design ensures that all session traces are automatically cleared when the browser tab is closed, preventing unauthorized access to credentials in shared or public browser environments.

Custom Toast Alert Engine (ToastProvider.jsx): Rather than importing heavy third-party notification libraries, the application implements a custom lightweight notification manager accessible throughout the component tree via a dedicated hook. The provider maintains a local stack of active alerts, each uniquely identified by a timestamp and

carrying a message payload along with a classification type such as success or error. Each alert is automatically unmounted after a fixed display duration and is rendered with background blur styling and entrance/exit keyframe animations to maintain visual consistency with the overall design language.

High-Fidelity PDF Export Engine: The application includes a dedicated PDF generation module that allows developers to export a structured report of the active scan session. The engine incorporates two notable technical provisions for internationalization. First, it integrates an Arabic text reshaping module to correctly render Arabic script in the exported document, with an internal fallback algorithm that maps standard Unicode Arabic characters to their correct contextual glyph forms — isolated, initial, medial, or final — when the external module is unavailable offline. Second, it dynamically retrieves an Arabic-compatible TrueType font from an open-source repository, converts it to a base64 representation, and registers it with the PDF engine's virtual file system to enable accurate right-to-left text rendering within the exported document.

3.X.5 API Integration, Security Headers, and Voice Uplink

The client interacts with the backend service through a centrally configured HTTP network layer that enforces strict rules for request interception and authorization.

Axios Base Client (apiClient.js): A shared HTTP client instance is configured with the backend service base URL drawn from environment variables, ensuring that the endpoint is never hardcoded and can be substituted across deployment environments without modifying application source code. Additional headers are included to bypass intermediate gateway inspection warnings that arise from the development tunneling infrastructure.

Bearer Token Authorization Interceptor: An outbound request interceptor automatically retrieves the active JSON Web Token (JWT) from session storage and injects it into the Authorization header of every outgoing HTTP request before it is dispatched. This mechanism ensures that no authenticated request can inadvertently be sent without the required security credential, and that the token injection logic remains centralized rather than duplicated across individual API call sites.

Secure WebRTC Voice Ingestion (RetellAgent.jsx): The platform implements a voice interaction interface using the WebRTC protocol through the Retell AI client SDK. The flow begins with the client requesting a secure, short-lived access token from the backend, which the backend generates by communicating with the Retell service using its stored credentials. The client then initializes an encrypted audio channel using this token, preventing any direct exposure of service credentials to the browser environment. The voice agent is presented within a floating, draggable interface element styled as a glass pill. The element responds to cursor proximity with interactive hover transitions, and an animated conic gradient border indicates live audio streaming status to the user.

3.X.6 Progressive Web App (PWA) and Offline Resilience

The front-end is configured as a fully compliant Progressive Web App (PWA) to ensure operational resilience under degraded or absent network conditions.

Service Worker Cache Strategies (sw.js): The application registers a service worker that applies distinct caching strategies to different categories of assets based on their update frequency and size characteristics.

Heavy three-dimensional assets, such as Spline scene files and binary geometry formats, are explicitly excluded from the service worker cache and are always fetched directly from the network. This prevents large binary files from consuming cache storage and causing stale rendering artifacts.

JavaScript and CSS bundles produced by the build tool are handled using a network-first strategy. The service worker always attempts to retrieve the latest compiled bundle from the network, updating the local cache on success, and falls back to the cached version only when the network is unavailable. This prevents stale bundle versions from causing blank-screen errors during development iteration cycles.

Core shell assets including the root HTML document and the PWA manifest are served from cache first, minimizing perceived startup latency for returning users.

Offline Background Sync and Request Queueing: If a user submits source code for scanning while the device is offline, the service worker intercepts the outbound POST request and places the full request payload — including the URL, HTTP method, headers, and body — into an in-memory queue rather than discarding it. The service worker subsequently registers a background synchronization task with the browser runtime. Once the device re-establishes a stable network connection, the browser triggers the sync event, at which point the service worker iterates through the queued requests and retransmits them to the backend sequentially, completing the scan without requiring any further action from the user.

Push Notification Integration: The service worker subscribes to push notification events to inform developers when a long-running background scan has completed. Incoming push notifications direct the user back to the visual analysis console interface, reducing the need for the user to keep the browser tab in focus during extended processing operations.

3.X.7 Third-Party Libraries

The following external dependencies are utilized within the client-side ecosystem to fulfill key functional and visual requirements:

Library Name	Version	Functional Role
react	^18.2.0	Declarative UI framework managing state and component lifecycles.
react-router-dom	^7.9.4	Client-side routing engine managing application navigation.
axios	^1.12.2	HTTP client handling async calls, request interception, and security headers.
framer-motion	^12.23.26	Handles smooth transitions, layout shifts, and 3D card tilt effects.
gsap	^3.13.0	Graphic timeline controls for ScrollTrigger, CustomCursor, and CardSwap.
three	^0.181.2	Core 3D engine handling WebGL objects, cameras, and lighting rendering.
@react-three/fiber	^8.18.0	React wrapper for declarative Three.js scene integration.
@splinetool/react-spline	^4.1.0	Integration library for loading interactive Spline 3D vector scenes.
jspdf	^4.2.1	Client-side PDF generation engine with custom TrueType font support.
retell-client-js-sdk	^2.0.7	WebRTC audio streaming client for the Retell AI voice assistant.
reactflow	^11.11.4	Node-based workflow rendering engine for data mapping diagrams.
react-markdown	^10.1.0	Render layer parsing raw AI markdown outputs into structured HTML nodes.
react-syntax-highlighter	^16.1.1	Decoupled syntax highlight tokenizer for secure code display rendering.
react-hot-toast	^2.6.0	Lightweight notification UI for active network state feedback.
ogl	^1.0.11	High-performance WebGL library for rendering custom GLSL fluid shaders.
@vercel/speed-insights	^1.3.0	Client-side performance auditing integration for Vercel deployment metrics.

3.X.8 Section References

[1] React Core Documentation. React Concurrent Rendering and Component Lifecycle. Available at: https://react.dev/

[2] Vite Building Engine. Vite Guide: Dynamic Bundling and Fast Module Replacement. Available at: https://vite.dev/

[3] Tailwind CSS. Tailwind CSS v4.0 Utility Framework Specification. Available at: https://tailwindcss.com/

[4] GreenSock. GSAP 3 Animation Library and ScrollTrigger Core Engine. Available at: https://gsap.com/

[5] Framer Motion. Declarative Animations and Layout Transitions for React. Available at: https://framer.com/motion/

[6] Mozilla Developer Network (MDN). Using CSS Custom Properties and DOM Theme Datasets. Available at: https://developer.mozilla.org/

[7] W3C Specification. Service Workers and Background Synchronization Protocols. Available at: https://www.w3.org/TR/service-workers/

[8] Spline 3D Design. Interactive 3D Web Integration and Runtime Assets. Available at: https://spline.design/

[9] React Three Fiber. Declarative Three.js in React Environments. Available at: https://docs.pmnd.rs/react-three-fiber

[10] OGL WebGL Framework. Minimal WebGL Library for Custom Fragment Shaders. Available at: https://github.com/vorg/ogl

[11] React API Reference. Code Splitting with React.lazy and Suspense. Available at: https://react.dev/reference/react/lazy

[12] GSAP Ticker API. RequestAnimationFrame Loop Sync and Custom Event Dispatches. Available at: https://gsap.com/docs/v3/GSAP/gsap.ticker/